

Documentação Técnica

Sistema de Personagens WoW

AEDS III — PUC Minas

Samuel Correia e Matheus Augusto

Algoritmos e Estruturas de Dados III

Pontifícia Universidade Católica de Minas Gerais

2026

1. Visão Geral

O Sistema de Personagens WoW é uma aplicação de gerenciamento de personagens temática do jogo World of Warcraft, desenvolvida em Python como trabalho prático da disciplina de Algoritmos e Estruturas de Dados III (AEDS III) da PUC Minas Lourdes.

O nosso sistema implementa estruturas de dados avançadas para persistência baseada em arquivos binários, incluindo Árvore B+, Hash Extensível e listas encadeadas em disco.

Característica	Detalhe
Linguagem	Python 3.x
Interfaces disponíveis	CLI (main.py) e Web HTTP (server.py, porta 8001)
Persistência	Arquivos binários com registros de tamanho fixo
Índices	Árvore B+ (disco) e Hash Extensível (disco)
Relacionamentos	1:N e N:N persistidos via listas encadeadas em arquivo
Grupos	Volátil — apenas em memória RAM

2. Arquitetura do Sistema

A aplicação segue um padrão MVC simplificado, organizado nos seguintes diretórios:

Camada	Diretório	Responsabilidade
Model	model/	Classes de entidade com serialização binária
DAO	dao/	Acesso a arquivos, gerenciamento de índices, CRUD
Controller	controller/	Implementações das estruturas de dados
View / Templates	templates/	Templates HTML com motor customizado
Dados	dados/	Arquivos binários de dados e índices em disco

2.1 Estrutura de Diretórios

```
AEDS3PersonagemdeWoW-main/
├── main.py           - Ponto de entrada CLI
├── server.py         - Servidor web (porta 8001)
├── model/
│   ├── Personagem.py
│   ├── Conta.py
│   ├── Bairro.py
│   ├── BairroPersonagem.py
│   ├── GrupoTemp.py
│   └── ArvoreBPlus.py - Implementação principal da Árvore B+
```

```

├── controller/
│   ├── HashExtensivel.py - Implementação do Hash Extensível
│   └── ArvoreBPlus.py    - Stub (não utilizado em runtime)
├── dao/
│   ├── ContaDAO.py
│   ├── PersonagemDAO.py
│   ├── BairroDAO.py
│   └── GrupoTempDAO.py
├── templates/           - HTML + CSS
└── dados/               - Arquivos binários de dados e índices

```

3. Entidades e Modelos de Dados

Todas as entidades persistidas utilizam registros de tamanho fixo em arquivo binário. O primeiro byte de cada registro é a lápide: espaço (' ') indica registro ativo; asterisco (*) indica exclusão lógica.

3.1 Personagem

Arquivo: model/Personagem.py | **Tamanho:** 51 bytes | **Formato:** <1s i 20s f i 10sq>

Campo	Tipo	Tamanho (bytes)	Descrição
lapide	char	1	Lápide: ' '=ativo, '*'=excluído
id	int32	4	Identificador único auto-incremento
nome	str	20	Nome do personagem (UTF-8, padding nulo)
nivel	float	4	Nível do personagem (ponto flutuante)
id_conta	int32	4	FK para Conta (relacionamento 1:N)
funcao	str	10	Função: 'dano', 'tanque' ou 'suporte'
prox	int64	8	Offset do próximo personagem da conta (lista 1:N)

3.2 Conta

Arquivo: model/Conta.py | **Tamanho:** 65 bytes | **Formato:** <1s i 20s 30s 10s>

Campo	Tipo	Tamanho (bytes)	Descrição
lapide	char	1	Lápide: ' '=ativo, '*'=excluído
id	int32	4	Identificador único auto-incremento
usuario	str	20	Nome de usuário (chave secundária)
email	str	30	Endereço de e-mail
data	str	10	Data de cadastro (DD/MM/AAAA)

3.3 Bairro

Arquivo: `model/Bairro.py` | Tamanho: 48 bytes | Formato: `<1s i 30s i q>`

Campo	Tipo	Tamanho (bytes)	Descrição
lapide	char	1	Lápide: ' '=ativo, '*'=excluído
id	int32	4	Identificador único auto-incremento
nome	str	30	Nome do bairro
id_dono	int32	4	FK para Personagem (dono do bairro)
reservado	int64	8	Campo reservado para uso futuro

3.4 BairroPersonagem

Arquivo: `model/BairroPersonagem.py` | Tamanho: 18 bytes | Formato: `<1s i i q>`

Tabela intermediária do relacionamento N:N entre Bairro e Personagem. O campo prox forma uma lista encadeada dentro do arquivo.

Campo	Tipo	Tamanho (bytes)	Descrição
lapide	char	1	Lápide: ' '=ativo, '*'=excluído
id_bairro	int32	4	FK para Bairro
id_personagem	int32	4	FK para Personagem
prox	int64	8	Offset do próximo par (bairro,personagem) com mesmo id_bairro

3.5 GrupoTemp

Arquivo: `model/GrupoTemp.py` | Apenas em memória | Formato: `<1s i i i>`

Entidade volátil armazenada exclusivamente em RAM via GrupoTempDAO. Não é persistida em disco e é perdida ao encerrar a aplicação.

Campo	Tipo	Descrição
lapide	char	Lápide de controle
id_grupo	int32	Identificador do grupo
id_conta	int32	FK para Conta do membro
id_personagem	int32	FK para Personagem do membro

4. Estruturas de Dados

4.1 Árvore B+ (model/ArvoreBPlus.py)

A Árvore B+ é a implementação principal de índice ordenado do sistema. É persistida em disco: cada nó ocupa um bloco de tamanho fixo no arquivo binário de índice.

Operações disponíveis

Método	Complexidade	Descrição
buscar(chave)	$O(\log n)$	Retorna o offset associado à chave, ou None
buscar_todos(chave)	$O(\log n + k)$	Retorna lista de todos os offsets com a chave (duplicatas)
inserir(chave, valor)	$O(\log n)$	Insere par (chave, offset) com split automático

Índices que utilizam Árvore B+

Arquivo de índice	Entidade	Chave indexada	Uso
index_bairro_id	Bairro	id	Busca primária de Bairro por ID
index_bairro_dono	Bairro	id_dono	Busca secundária: bairros de um dono (retorna ordenado)
index_perso_id	Personagem	id	Busca secundária por ID via B+
index_perso_nivel	Personagem	nivel	Busca por nível do personagem

4.2 Hash Extensível (controller/HashExtensivel.py)

O Hash Extensível permite busca $O(1)$ e cresce dinamicamente dobrando o diretório quando um bucket transborda. Persiste o diretório e os buckets em arquivos binários separados.

Parâmetro	Valor
Arquivo diretório	<code>nome_arquivo + '_dir.bin'</code>
Arquivo buckets	<code>nome_arquivo + '_buckets.bin'</code>
Formato do bucket	<code><ii (iq iq iq iq)></code>

Estrutura do Bucket

Campo	Tipo	Descrição
prof_local	int32	Profundidade local do bucket
qtd	int32	Quantidade de entradas presentes

chave[0..3]	int32	Chaves armazenadas (inteiro)
end[0..3]	int64	Endereços (offsets) associados a cada chave

Operações disponíveis

Método	Descrição
search(chave)	Retorna o offset associado, ou None
insert(chave, end)	Insere ou atualiza o offset; split + duplicação de diretório se bucket cheio
remove(chave)	Remove a entrada do bucket compactando o vetor

Índices que utilizam Hash Extensível

Arquivo de índice	Chave	Uso
index_contas	id (int)	Índice primário de Conta
index_contas_usuario	usuario (str)	Índice secundário por nome de usuário
index_personagem_id	id (int)	Índice primário de Personagem (O(1))
index_relacao_conta_perso	id_conta (int)	Topo da lista 1:N Conta→Personagem
index_bairro_personagem	id_bairro (int)	Topo da lista N:N Bairro→Personagem

5. Camada de Acesso a Dados (DAOs)

5.1 ContaDAO

Aspecto	Detalhe
Arquivo de dados	<code>dados/contas.bin</code>
Tamanho do registro	65 bytes
Índice primário	HashExtensivel (por ID)
Índice secundário	HashExtensivel (por usuario)
Exclusão	Lógica — marca lápide '*'
Ordenação externa	Intercalação balanceada por usuario → <code>contas_ordenadas.bin</code>

Métodos principais

Método	Descrição
create(conta)	Gera ID, grava registro, atualiza ambos os hashes
read(id)	Busca via hash primário (O(1))

read_por_usuario(nome)	Busca via hash secundário
update(id, conta)	Sobrescreve registro; atualiza hashes se usuario mudou
delete(id)	Marca lápide; remove dos dois hashes
ordenar_externo_usuario()	Cria runs ordenados e intercala em arquivo final

5.2 PersonagemDAO

Aspecto	Detalhe
Arquivo de dados	<code>dados/personagens.bin</code>
Tamanho do registro	51 bytes
Índice primário	HashExtensivel hash_id (busca O(1))
Índice 1:N	HashExtensivel hash_relacao: id_conta → offset do 1º personagem
Índice B+ por ID	ArvoreBPlus arvore_id
Índice B+ por nível	ArvoreBPlus arvore_nivel
Lista 1:N	Campo Personagem.prox encadeia personagens da mesma conta

Métodos principais

Método	Descrição
create(personagem)	Grava registro; atualiza hash_id, hash_relacao (topo da lista 1:N), arvore_id, arvore_nivel
read(id)	Busca via hash_id com fallback sequencial
read_bplus(id)	Busca via arvore_id
listar_por_conta(id_conta)	Percorre lista encadeada a partir do hash_relacao
buscar_por_nivel(nivel)	Busca via arvore_nivel
update(id, ...)	Regrava registro preservando Personagem.prox
delete(id)	Marca lápide, remove do hash_id, chama remover_relacoes_por_personagem()

5.3 BairroDAO

Aspecto	Detalhe
Arquivo de dados	<code>dados/bairros.bin</code>
Arquivo N:N	<code>dados/bairro_personagem.bin</code>
Índice primário	ArvoreBPlus arvore_id (por ID do bairro)
Índice secundário	ArvoreBPlus arvore_dono (por id_dono) — resultado ordenado por bairro.id

Índice N:N	HashExtensivel hash_relacao: id_bairro → offset do 1º BairroPersonagem
Lista N:N	Campo BairroPersonagem.prox encadeia os pares do mesmo bairro

Métodos principais

Método	Descrição
create(bairro)	Grava registro; insere em arvore_id e arvore_dono
read(id)	Busca via arvore_id com fallback sequencial
buscar_por_dono(id_dono)	Busca via arvore_dono; retorna lista ordenada por bairro.id
adicionar_personagem(ib, ip)	Verifica duplicata via _relacao_existe(), acrescenta ao arquivo N:N, atualiza hash_relacao
remover_personagem(ib, ip)	Marca lápide no registro N:N
delete(id)	Exclusão lógica + cascade: chama remover_todas_relacoes_do_bairro()
remover_relacoes_por_personagem(ip)	Marca todos os BairroPersonagem com o personagem; reconstrói apontadores

5.4 GrupoTempDAO

Gerencia grupos voláteis (somente RAM). Os grupos são perdidos ao encerrar a aplicação.

Regra de composição	Limite
Tanque	1 por grupo
Suporte	1 por grupo
Dano	3 por grupo
Total	5 membros
Por conta	1 personagem por conta por grupo

6. Arquivos Binários e Índices

Todos os arquivos são armazenados no diretório dados/.

6.1 Arquivos de Dados

Arquivo	Estrutura	Tamanho/registro
contas.bin	Cabeçalho 4B (último ID) + registros	65 bytes
personagens.bin	Cabeçalho 4B (último ID) + registros	51 bytes

bairros.bin	Cabeçalho 4B (último ID) + registros	48 bytes
bairro_personagem.bin	Sem cabeçalho, append-only	18 bytes

6.2 Arquivos de Índice — Hash Extensível

Par de arquivos	Índice
index_contas_dir.bin / _buckets.bin	Hash primário de Conta (por ID)
index_contas_usuario_dir.bin / _buckets.bin	Hash secundário de Conta (por usuario)
index_personagem_id_dir.bin / _buckets.bin	Hash primário de Personagem (por ID)
index_relacao_conta_perso_dir.bin / _buckets.bin	Topo da lista 1:N Conta → Personagem
index_bairro_personagem_dir.bin / _buckets.bin	Topo da lista N:N Bairro → Personagem

6.3 Arquivos de Índice — Árvore B+

Arquivo de índice	Entidade	Campo indexado	Tipo de busca
index_bairro_id	Bairro	id	Primária — busca por ID
index_bairro_dono	Bairro	id_dono	Secundária — bairros de um dono (ordenado)
index_perso_id	Personagem	id	Secundária — busca por ID via B+
index_perso_nivel	Personagem	nivel	Secundária — busca por nível

7. Interface Web

O servidor web é implementado exclusivamente com a biblioteca padrão do Python (http.server), sem frameworks externos. Usa um motor de templates customizado com suporte a herança, blocos, includes e condicionais.

7.1 Motor de Templates

Recurso	Sintaxe
Herança	{% extends "base.html" %}
Bloco	{% block nome %} ... {% endblock %}
Include	{% include "parcial.html" %}
Condicional	{% if variavel %} ... {% else %} ... {% endif %}
Variável	{{ variavel }}
Variável sem escape	{{ variavel safe }}

8. Interface CLI

A interface de linha de comando é iniciada executando main.py. Organiza-se em menus hierárquicos.

8.1 menu_contas() — Menu Principal

Opção	Ação	Estrutura utilizada
1	Criar Nova Conta	Arquivo sequencial + Hash primário e secundário
2	Pesquisar Conta por ID	Hash Extensível (O(1))
3	Atualizar Dados da Conta	Hash primário + Hash secundário
4	Excluir Conta (lógica + hash)	Lápide + remoção dos índices
5	Entrar (Login)	Hash por usuário
6	Ordenação Externa por Usuário	Intercalação balanceada em disco
7	Sair	—

8.2 menu_personagens() — Após Login

Opção	Ação	Estrutura utilizada
1	Criar Personagem	Hash primário + 1:N + Árvore B+
2	Pesquisar por ID (Hash)	Hash Extensível (O(1))
3	Pesquisar por ID (Árvore B+)	Árvore B+
4	Pesquisar por Nível (B+)	Árvore B+ arvore_nivel
5	Listar Meus Personagens	Hash 1:N + lista encadeada
6	Atualizar Personagem	Hash primário
7	Excluir Personagem (lógica)	Lápide + cascade N:N
8	Gerenciar Grupos	GrupoTempDAO (RAM)
9	Gerenciar Bairros (N:N)	BairroDAO + Hash + Árvore B+
10	Logout / Voltar	—

8.3 menu_bairros() — Gerenciamento N:N

Opção	Ação	Estrutura utilizada
1	Criar Novo Bairro	Árvore B+ (id e dono)
2	Listar Todos os Bairros	Varredura sequencial

3	Buscar por ID (Árvore B+)	Árvore B+ arvore_id
4	Buscar por Dono (Árvore B+, ord.)	Árvore B+ arvore_dono + sort por id
5	Adicionar Personagem ao Bairro	Hash N:N + append em bairro_personagem.bin
6	Remover Personagem do Bairro	Lápide no registro N:N
7	Listar Personagens do Bairro	Hash N:N + lista encadeada
8	Atualizar Bairro	Árvore B+ + arquivo bairros.bin
9	Excluir Bairro	Lápide + cascade (remover_todas_relacoes)
10	Voltar	—

9. Relacionamentos entre Entidades

9.1 Conta 1:N Personagem

Cada conta pode possuir múltiplos personagens. O relacionamento é mantido por uma lista encadeada cujos nós são os próprios registros no arquivo personagens.bin, usando o campo prox como ponteiro de offset.

Componente	Detalhe
Índice de acesso	HashExtensivel hash_relacao: id_conta → offset do 1º Personagem
Encadeamento	Personagem.prox → offset do próximo Personagem da mesma conta (-1 = fim)
Inserção	Novo personagem inserido no topo da lista; hash_relacao atualizado
Listagem	PersonagemDAO.listar_por_conta() percorre a cadeia pelo campo prox

9.2 Personagem N:N Bairro

Um personagem pode pertencer a múltiplos bairros, e um bairro pode ter múltiplos personagens. O relacionamento é mantido por registros na tabela intermediária bairro_personagem.bin, encadeados por id_bairro.

Componente	Detalhe
Arquivo	bairro_personagem.bin (append-only, 18 bytes/registro)
Índice de acesso	HashExtensivel hash_relacao: id_bairro → offset do 1º BairroPersonagem
Encadeamento	BairroPersonagem.prox → offset do próximo par com mesmo id_bairro (-1 = fim)
Verificação	BairroDAO._relacao_existe() evita duplicatas antes de inserir
Cascade delete	delete(bairro) → remover_todas_relacoes_do_bairro(); delete(personagem) → remover_relacoes_por_personagem()

9.3 Personagem N:N GrupoTemp (RAM)

Grupos são associações temporárias entre personagens de contas distintas, gerenciadas inteiramente em memória pela classe GrupoTempDAO. As restrições de composição (1 tanque, 1 suporte, 3 danos, máx. 5 membros) são validadas no momento da inserção.

Glossário

Termo	Definição
Lápide (tombstone)	Byte de controle no início de cada registro. Espaço (' ') = ativo; asterisco ('*') = excluído logicamente.
Offset	Posição em bytes a partir do início do arquivo binário, usada como endereço direto de acesso (seek).
Registro de tamanho fixo	Estrutura com número de bytes determinístico, permitindo acesso direto (posição = cabeçalho + n × tamanho_registro).
Hash Extensível	Estrutura de índice dinâmica que cresce dobrando o diretório quando um bucket atinge capacidade máxima.
Árvore B+	Árvore balanceada de busca onde todos os dados ficam nas folhas, encadeadas para permitir varreduras ordenadas.
Split	Divisão de um nó/bucket cheio em dois, redistribuindo os elementos.
1:N	Relacionamento onde um registro pai possui múltiplos registros filhos.
N:N	Relacionamento onde múltiplos registros de uma entidade se associam a múltiplos de outra.
DAO	Data Access Object — padrão que encapsula a lógica de acesso a dados separando-a das regras de negócio.

TP3 - Formulário

Questões:

1.

Relacionamento N:N entre: Bairro, Personagem com a tabela intermediária sendo BairroPersonagem
(BairroPersonagem conecta id_bairro e id_personagem)

2.

Estrutura de índice utilizada e justificativas:

BairroDAO usa:

Árvore B+ para id_bairro e para id_dono

PersonagemDAO usa:

Hash Extensível para índice primário de id_personagem

Hash Extensível para relacionamento 1:N id_conta -> primeiro personagem

Árvore B+ para índice por id_personagem e nivel

Justificativa:

Hash Extensível serve bem para acesso direto e para mapear chaves a offsets.

B+ serve bem para buscas ordenadas e consultas secundárias (id_dono, nivel).

3.

Chave composta da tabela intermediária

Implementada na tabela intermediária BairroPersonagem

A combinação lógica é:

id_bairro

id_personagem

Não há um constraint de banco relacional, mas o código checa duplicatas em

BairroDAO._relacao_existe(...)

4.

Busca eficiente por meio do índice

Para bairros: -BairroDAO.read(id) usa arvore_id.buscar(id) para encontrar o offset(deslocamento) no arquivo -BairroDAO.buscar_por_dono(id_dono) usa arvore_dono.buscar_todos(id_dono)

Para membros do bairro: -hash_relacao mapeia id_bairro -> offset do primeiro BairroPersonagem -BairroPersonagem.prox forma uma lista encadeada de relações daquele bairro

Para personagens: -PersonagemDAO.read(id) usa hash_id.search(id) para acesso O(1)

-listar_por_conta usa hash 1:N e segue Personagem.prox

5.

Integridade referencial e cascata

Há cascata em exclusão: -PersonagemDAO.delete(id) marca o personagem e chama:

BairroDAO.remover_relacoes_por_personagem(id) -BairroDAO.delete(id) marca o bairro e chama:

BairroDAO.remover_todas_relacoes_do_bairro(id)

Isso garante que relacionamentos N:N sejam marcados como excluídos em cascata.

6.

Persistência dos dados da tabela intermediária

BairroPersonagem.bin armazena registros fixos do relacionamento

Cada registro tem:

lapide (1 byte)

id_bairro (4 bytes)

id_personagem (4 bytes)

prox (8 bytes)

O arquivo de relacionamento não usa cabeçalho de ID sequencial; ele é apenas um arquivo de registros append com índice externo em hash.

7.

Integração do código da tabela intermediária com o CRUD

BairroDAO.delete(...) limpa os relacionamentos N:N daquele bairro

PersonagemDAO.delete(...) aciona remoção das relações em que aquele personagem participa.

Assim, o CRUD principal de Bairro e Personagem está integrado com a tabela intermediária pelo: gerenciamento de relações , exclusões em cascata, leitura das relações a partir do hash_relacao.

LINKS:

youtube: <https://youtu.be/OQ7RscZaqlg>

GitHub: <https://github.com/samulkil/AEDS3PersonagemdeWoW.git>